SECURITY



An Introduction to Attesting Hitlessly Updatable Systems

Matt King

Principal Security Architect, NVIDIA



Disclaimers

- Examples are generic, no specific formats are being shown or recommended
  - Standards for evidence and reference formats are evolving
  - Different attesters/verifiers have different requirements for how evidence and references are delivered
- It is assumed the attester produces accurate and complete evidence
  - Flaws in evidence generation are almost always fatal to attestation, and so are out of scope for this presentation
  - Cryptographic methods for proving authenticity and integrity are assumed, but not shown

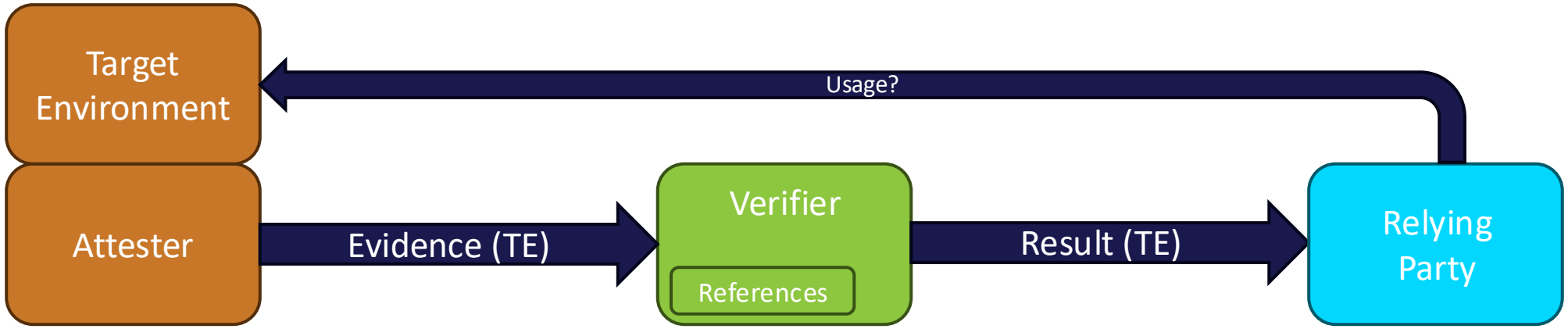
Background – Hitless Updates

- Activation of new firmware or software without preventing forward progress of the running workload

- A crucial aspect of maintaining system security is routine updates. By minimizing downtime, the ability to hitlessly apply updates ensures security fixes can be deployed regularly and rapidly

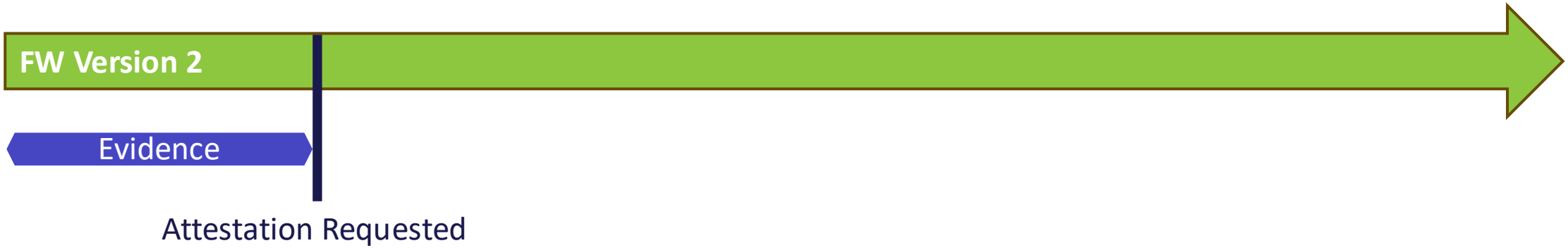
- AKA: Seamless Updates, Impactless Updates, Runtime Updates, In-service Updates

Background – Attestation Oversimplified



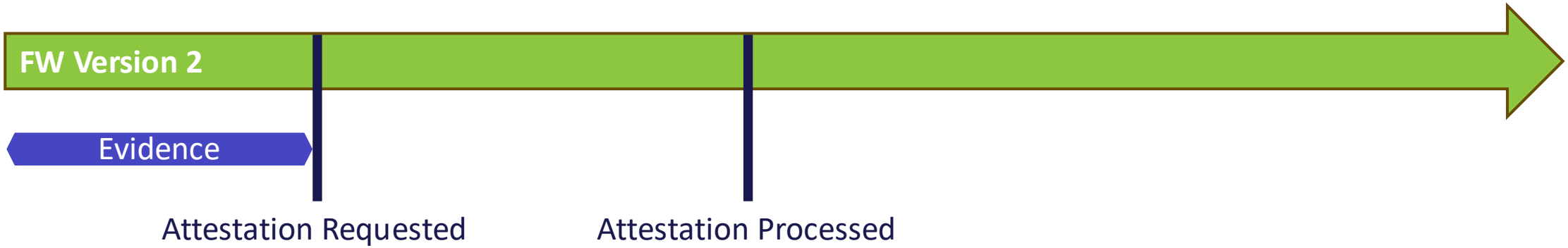
- See RATS Architecture (RFC 9334) for more details
- A relying party can only draw conclusions about a TE that are supported by the available evidence
  - So, what does the relying party want to know about the TE?

Evidence for Non-Updatable Runtimes



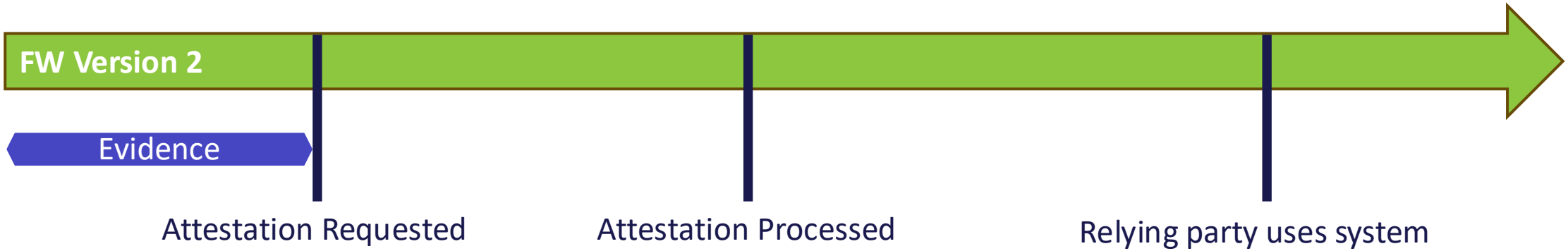
- Evidence about a TE is collected at a single point in time
  - But if the TE is not changing, evidence should be the same regardless of when it was collected
- Provides information about the state and configuration of the TE
  - Component Identifiers (e.g. vendor, model, version)
  - Measurements (e.g. hashes, SVN, debug-enabling status)

Verifying Non-Updatable Runtimes



- A gap exists between attestation and evaluation/use of the TE
  - Evidence doesn't cover the state of the TE after it was produced
- In a system that requires a reset to active an update, the attested TE is static
  - Provides an implied invariant that verifiers and relying parties can reason about
  - Caveat: verifier must be able to confirm from the evidence that hitless updates are blocked

Evaluating Non-Updatable Runtimes



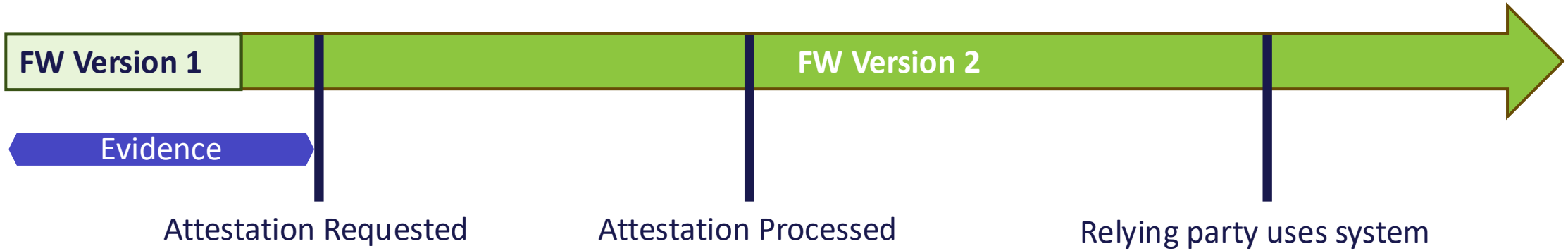
- Relying party can then trust the TE enforces non-updatability
  - The environment at the time of use will match the attested evidence
- Relying party can have confidence that the TE is operating in the expected state
  - As described by the reference measurements

Evidence for Updatable Components



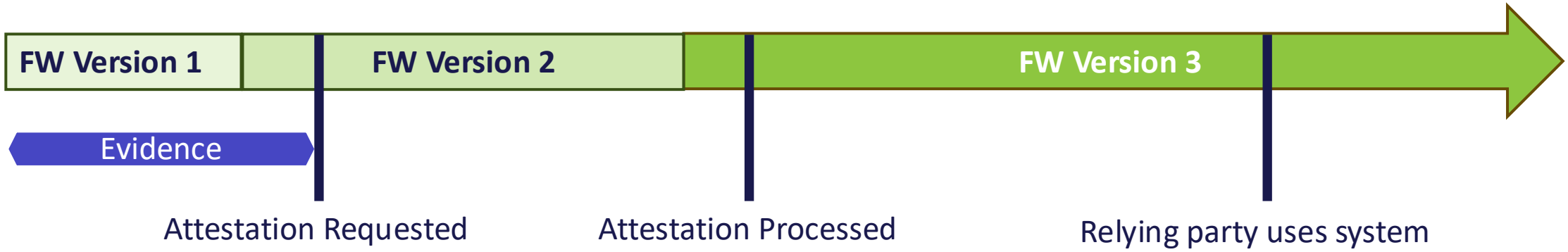
- In an updatable system, the TE can change.
- Evidence needs to capture prior states, in addition to the current state.
- Additional evidence needed
  - History (e.g. update log)

Evaluating Updatable Components



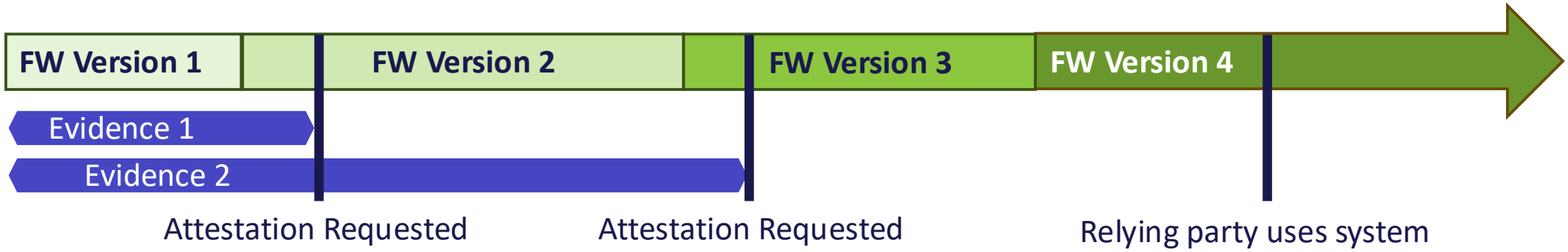
- If the TE cannot change after attestation, verification results and relying party decisions continue to work the same as in the static case
  - The environment at the time of use will match the attested evidence
  - Relying party has confidence that the TE is operating in the state described by the reference measurements
- History evidence will be more work for the verifier to analyze though

Gap with Hitlessly Updatable Components



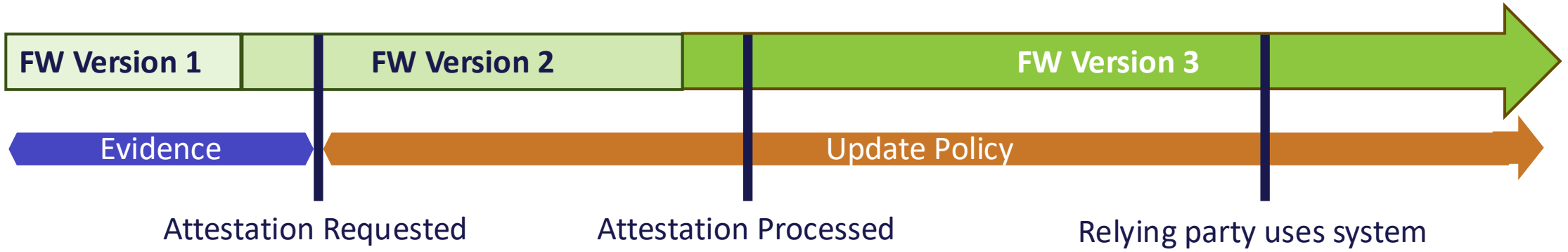
- However, the TE can change between attestation and usage
- Verifier can evaluate the history up until the point the evidence was created
  - But cannot know the state of the system at the time relying party uses it

Gap with Hitlessly Updatable Components



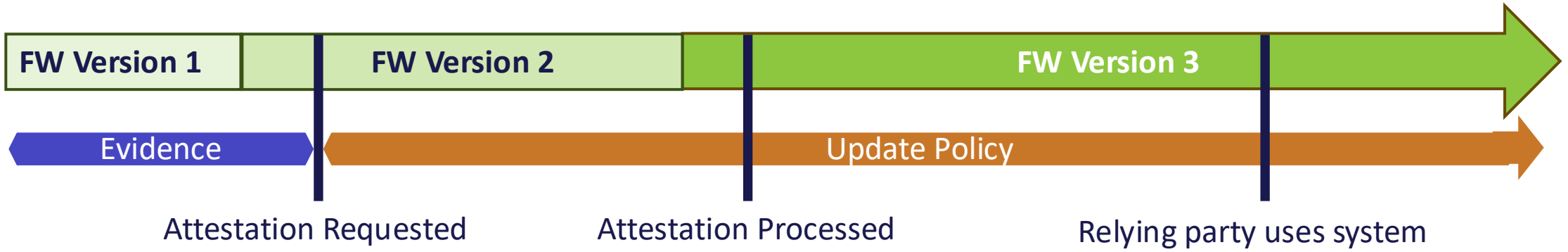
- Generating fresh evidence doesn't address this gap
  - The new evidence will also become stale
- Evidence capturing only present and past state is insufficient when the TE can change after that evidence is created

Evidence for Hitlessly Updatable Components



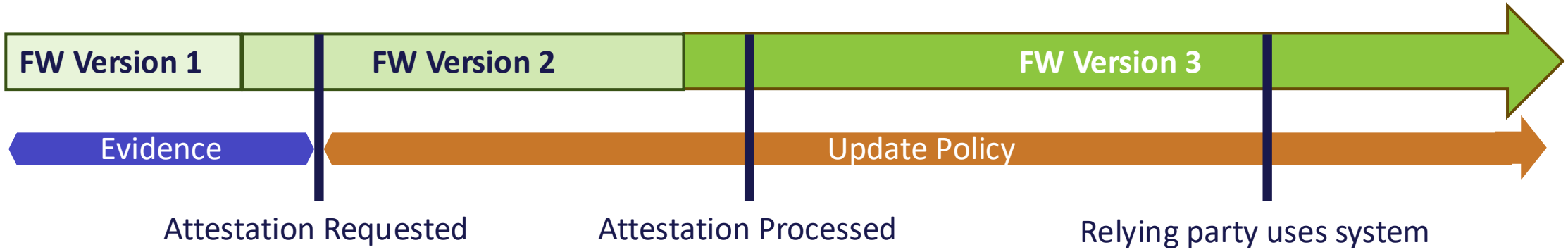
- An invariant is needed to describe future changes allowed after attestation
- Update Policy (e.g. anti-rollback, secondary authorization public key)
  - Communicates limits on future state that the relying party can reason about
- If the current state (reported in the evidence) is trustworthy, then the TE can be trusted to enforce the update policy

Verifying Hitlessly Updatable Components



- Verification of an updatable system against references can validate
  - Prior versions upon which the TE that produced the evidence relies, and
  - The currently running target environment (which enforces the update policy)
- Either the verifier or relying party will also need to understand and interpret the update policy for future changes to the TE

Evaluating Hitlessly Updatable Components



- Both results and update policy should be evaluated to make a decision
- Relying party must refuse TEs with unacceptable update policies
  - Just like they would refuse a TE with a known-vulnerable firmware
- Alleviates the need to notify relying party prior to subsequent updates
  - They have already accepted the update policy

Changes for Hitlessly Updatable Components

Evidence for static components:

- Component Identifiers
- *Measurements - invariant*

Evidence for updatable components:

- Component Identifiers
- *Measurements*
- *History*
- *Update Policy - invariant*

- History is added to the evidence to capture the additional dependencies of dynamic target environments
- A new invariant (update policy) is added to replace measurements, which are no longer static
  - Provides assurance about future state lost by making measurements changeable

Limits on Attestation for Updatable Components

Evidence for static components:

- Component Identifiers
- *Measurements - invariant*

Evidence for updatable components:

- Component Identifiers
- *Measurements*
- *History*
- *Update Policy - invariant*

- Update policy must be invariant until reset
  - Policy cannot itself be updated as part of a hitless update, otherwise there are no guarantees about future behavior
- History complicates storage, integrity, and verification of evidence
  - Has a potentially unbounded size and must be carefully managed
- Future updates can introduce bugs instead of fixing them

Call to Action

- Ensure your hitlessly updatable devices and systems produce sufficient evidence
- Prepare verifiers and relying parties for handling the additional evidence produced by hitlessly updatable devices
- Share policy use-cases for your devices and systems
- Participate in standards groups defining evidence, references, and verifier behavior
  - IETF, TCG, DMTF, OCP, etc.

Thank You!

